

Reinforcement Learning

ECMM422

Fabrizio Costa

Winter 2026



Types of learning

Supervised learning

In this paradigm, the system is provided with correct examples of the mapping between observations and the target variables (categorical or scalar).

Unsupervised learning

In this paradigm, the system is only provided with observation data and attempts to extract patterns in the signal.

Reinforcement learning

The system can interact with its environment (eg, a robot) and attempts to optimise a **long term goal** from **local observations**.

Reinforcement Learning

Lack of supervision

The system is not given training examples of correct actions or how to solve the task.

Closed loop systems

The system can affect its environment by its actions, and therefore **its future inputs**.

Learning from interaction

The system learns to improve its decision taking from the consequences of its actions, maximising a reward function.

Delayed payback

Consequences of the system's choices and actions become evident only much later, including reward signal.

Textbook: *Reinforcement Learning: an Introduction*. Richard S. Sutton and Andrew G. Barto, MIT press, 1998.

Outline

- 1 The k -armed bandit
- 2 Dynamic Programming
 - Closed loop systems
 - Markov decision processes
 - Estimating value functions
- 3 Temporal-difference learning
 - On-policy learning: SARSA
 - Off-policy learning: Q-learning
- 4 Deep reinforcement learning

The k -armed bandit problem

- ▶ Imagine a slot machine with k arms.
- ▶ Each arm pays-off with a different (unknown) probability.
- ▶ The only action allowed at each round is to pull one and only one lever (out of k), and collect the reward.
- ▶ The agent doesn't know the arms pay-off a priori, and aims to maximise payoff over 1000 pulls.

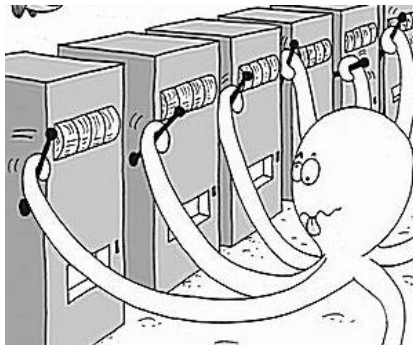


Image source: Microsoft Research.

The k -armed bandit problem

- ▶ We denote the **action** taken at time t as a_t .
- ▶ The **reward** R_t of an action a is the amount of money payed by the machine at this round t .

The k -armed bandit problem

- ▶ We denote the **action** taken at time t as a_t .
- ▶ The **reward** R_t of an action a is the amount of money payed by the machine at this round t .
- ▶ the **value** of an action a is the expected pay-off $q_*(a) = \mathbb{E}[R_t | A_t = a]$. If this value was known, solving the problem would be trivial: just pick the level $\operatorname{argmax}_a q_*(a)$.
- ▶ Otherwise, the current value **estimate** at time t is denoted by $Q_t(a)$.

The k -armed bandit problem

- ▶ We denote the **action** taken at time t as a_t .
- ▶ The **reward** R_t of an action a is the amount of money payed by the machine at this round t .
- ▶ the **value** of an action a is the expected pay-off $q_*(a) = \mathbb{E}[R_t | A_t = a]$. If this value was known, solving the problem would be trivial: just pick the level $\operatorname{argmax}_a q_*(a)$.
- ▶ Otherwise, the current value **estimate** at time t is denoted by $Q_t(a)$.

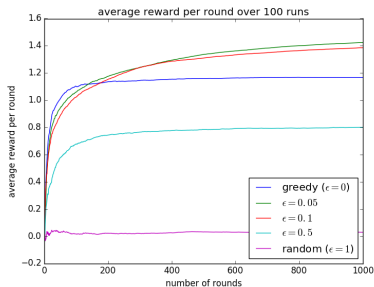
Hence, at each time t , we have two possible course of action:

- ▶ Choose the lever **maximising estimated pay-off** ($A_t = \operatorname{argmax}_a Q_t(a)$). This is called **exploitation** (but the estimate could be wrong, and will stay this way).
- ▶ Choose another lever to try and **improve your value estimates**. This is called **exploration**.

- ▶ One solution is called the ϵ -greedy strategy.
- ▶ It selects an action as follows

$$\begin{cases} a_t = \operatorname{argmax}_a Q_t(a) & \text{with prob } 1 - \epsilon \\ \text{a random action} & \text{with prob } \epsilon \end{cases}$$

- ▶ when $\epsilon = 0$ the system is using the greedy strategy.
- ▶ when $\epsilon = 1$ the system is always choosing a random lever (fully random strategy).



Upper-Confidence-Bound action selection

ϵ -greedy strategy forces the system to try a priori sub-optimal actions, but select them indiscriminatively.

It would be better to select non-greedy actions based on their **potential** of being optimal.

Upper-Confidence-Bound action selection

ϵ -greedy strategy forces the system to try a priori sub-optimal actions, but select them indiscriminatively.

It would be better to select non-greedy actions based on their **potential** of being optimal.

We can model this potential as:

$$A_t = \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

where $N_t(a)$ is the number of time action a has been selected

Upper-Confidence-Bound action selection

ϵ -greedy strategy forces the system to try a priori sub-optimal actions, but select them indiscriminatively.

It would be better to select non-greedy actions based on their **potential** of being optimal.

We can model this potential as:

$$A_t = \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

where $N_t(a)$ is the number of time action a has been selected

- ▶ $c > 0$ determines the **degree of exploration**.

Upper-Confidence-Bound action selection

ϵ -greedy strategy forces the system to try a priori sub-optimal actions, but select them indiscriminatively.

It would be better to select non-greedy actions based on their **potential** of being optimal.

We can model this potential as:

$$A_t = \operatorname{argmax}_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

where $N_t(a)$ is the number of time action a has been selected

- ▶ $c > 0$ determines the **degree of exploration**.
- ▶ if $N_t(a) = 0$ then action a is deemed to have **maximum potential** (it has never been tried before!).

Upper-Confidence-Bound action selection

ϵ -greedy strategy forces the system to try a priori sub-optimal actions, but select them indiscriminatively.

It would be better to select non-greedy actions based on their **potential** of being optimal.

We can model this potential as:

$$A_t = \underset{a}{\operatorname{argmax}} \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

where $N_t(a)$ is the number of time action a has been selected

- ▶ $c > 0$ determines the **degree of exploration**.
- ▶ if $N_t(a) = 0$ then action a is deemed to have **maximum potential** (it has never been tried before!).
- ▶ Each time an action a is chosen, **its potential is reduced** w.r.t. all other actions.

Outline

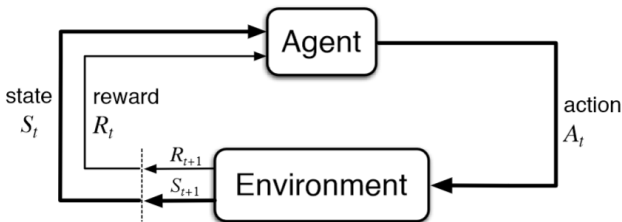
- 1 The k -armed bandit
- 2 Dynamic Programming
 - Closed loop systems
 - Markov decision processes
 - Estimating value functions
- 3 Temporal-difference learning
 - On-policy learning: SARSA
 - Off-policy learning: Q-learning
- 4 Deep reinforcement learning

The Agent-Environment interface

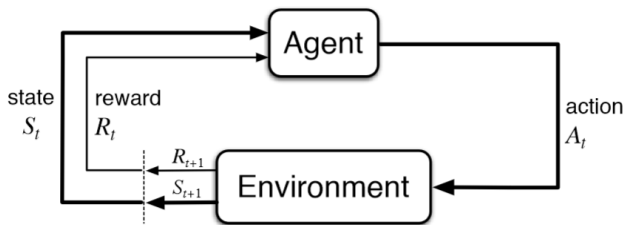
The learner and decision maker is called the **agent**. The thing it interacts with, comprising of everything outside the agent, is called the **environment**. These interact continuously, the agent selecting actions and the environment responding to those actions and presenting **new situations** to the agent.

The environment also gives rise to **rewards**, special numerical values that the agent tries to maximize over time. A complete specification of an environment, including how rewards are determined, defines a **task**, one instance of the reinforcement learning problem.

(Sutton & Barto, 2016)

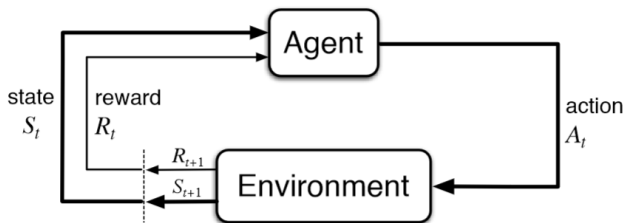


Definitions



- ▶ The agent and environment **interact** at discrete time steps $t = 0, 1, 2, \dots$
- ▶ At each time step t , the agent receives some **representation of the environment state** $S_t \in \mathcal{S}$ and **selects an available action** $A_t \in \mathcal{A}(S_t)$.
- ▶ At then next step, the agent **receives numerical rewards** for previous actions $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ and finds itself in a new state S_{t+1} .

Definitions



- ▶ At each time step, the agent implements a probabilistic mapping from states to actions called a **policy** π_t , where $\pi_t(a|s)$ is **the probability to select action a when in state s** .
- ▶ reinforcement learning specifies how the agent **learns and updates its policy** π_t as a result of experience.

Loosely speaking, the agent's aim is to maximise reward **in the long run**.

Rewards R_t

At each step, the agent receives a single scalar reward $R_t \in \mathbb{R}$. The reward is the way the **agent's goal** is encoded.

Rewards and returns

Rewards R_t

At each step, the agent receives a single scalar reward $R_t \in \mathbb{R}$. The reward is the way the **agent's goal** is encoded.

Returns G_t

The agent is tasked to maximise not the instantaneous reward, but rather the **expected return** in the long run

For an **episodic task**:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T$$

Rewards and returns

Rewards R_t

At each step, the agent receives a single scalar reward $R_t \in \mathbb{R}$. The reward is the way the **agent's goal** is encoded.

Returns G_t

The agent is tasked to maximise not the instantaneous reward, but rather the **expected return** in the long run

For an **episodic task**:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T$$

For **continuing tasks**, where $T = \infty$, this return can be infinite. In practice, for this reason and to account for uncertainty of future rewards, the return is usually discounted:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1}$$

The Markov property

Definition

We can define the probability for the future state and reward as a joint probability of all states, actions, rewards that have happened:

$$p[S_{t+1} = s, R_{t+1} = r | S_0, A_0, R_1, \dots, S_{t-1}, A_{t-1}, R_t, S_t, A_t]$$

If the state signal has the **Markov property** then the environment response at time $t + 1$ depends only on the **state** and **action** at time t :

$$p[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a] = p[s', r | s, a]$$

In other words, Markovian systems are **memory-less** systems. This assumption is so powerful that it is worth making even for systems that are not strictly Markovian.

Note that some history can be encoded in the state vector: eg, encoding position, speed and acceleration.

Markov decision processes (MDPs)

The state/action/reward agent we have been discussing can be probabilistically modelled as a Markov decision process (MDP):

- ▶ The **state-reward probability** is given by:

$$p[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a] = p[s', r | s, a]$$

Markov decision processes (MDPs)

The state/action/reward agent we have been discussing can be probabilistically modelled as a Markov decision process (MDP):

- ▶ The **state-reward probability** is given by:

$$p[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a] = p[s', r | s, a]$$

- ▶ The **expected rewards** for a state-action pair are:

$$r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p[s', r | s, a]$$

Markov decision processes (MDPs)

The state/action/reward agent we have been discussing can be probabilistically modelled as a Markov decision process (MDP):

- ▶ The **state-reward probability** is given by:

$$p[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a] = p[s', r | s, a]$$

- ▶ The **expected rewards** for a state-action pair are:

$$r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p[s', r | s, a]$$

- ▶ The **state-transition probabilities** are given by:

$$p[s' | s, a] = p[S_{t+1} = s' | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} p[s', r | s, a]$$

Markov decision processes (MDPs)

The state/action/reward agent we have been discussing can be probabilistically modelled as a Markov decision process (MDP):

- ▶ The **state-reward probability** is given by:

$$p[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a] = p[s', r | s, a]$$

- ▶ The **expected rewards** for a state-action pair are:

$$r(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p[s', r | s, a]$$

- ▶ The **state-transition probabilities** are given by:

$$p[s' | s, a] = p[S_{t+1} = s' | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} p[s', r | s, a]$$

- ▶ The **expected rewards** for a initial-state/action/next-state triplet:

$$r(s, a, s') = \mathbb{E}[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s'] = \frac{\sum_{r \in \mathcal{R}} r \cdot p[s', r | s, a]}{p[s' | s, a]}$$

A value function is an estimate of how good are individual states—in terms of future rewards. For a given policy $\pi(a|s)$, we can define $v_\pi(s)$ as:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right]$$

Value functions

A value function is an estimate of how good are individual states—in terms of future rewards. For a given policy $\pi(a|s)$, we can define $v_\pi(s)$ as:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right]$$

Similarly, the value of taking action a when in state s is:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right]$$

Value functions

A value function is an estimate of how good are individual states—in terms of future rewards. For a given policy $\pi(a|s)$, we can define $v_\pi(s)$ as:

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right]$$

Similarly, the value of taking action a when in state s is:

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s, A_t = a \right]$$

Given optimal value functions v_* and q_* , the optimal policy is trivial: it boils down to choosing the best value action at each step.

Value function: recursive formulation

Importantly, the value function can be calculated recursively using the law of iterated expectation:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

Value function: recursive formulation

Importantly, the value function can be calculated recursively using the law of iterated expectation:

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | S_t = s] \\&= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right]\end{aligned}$$

Value function: recursive formulation

Importantly, the value function can be calculated recursively using the law of iterated expectation:

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | S_t = s] \\&= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right] \\&= \mathbb{E}_{\pi} \left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] = \dots\end{aligned}$$

Value function: recursive formulation

Importantly, the value function can be calculated recursively using the law of iterated expectation:

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | S_t = s] \\&= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right] \\&= \mathbb{E}_{\pi} \left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] = \dots \\&= \sum_a \pi(a|s) \left(r(s, a) + \gamma \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] \right) = \dots\end{aligned}$$

Value function: recursive formulation

Importantly, the value function can be calculated recursively using the law of iterated expectation:

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | S_t = s] \\&= \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s \right] \\&= \mathbb{E}_{\pi} \left[R_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] = \dots \\&= \sum_a \pi(a|s) \left(r(s, a) + \gamma \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+2} | S_t = s \right] \right) = \dots \\&= \sum_a \pi(a|s) \left(r(s, a) + \gamma \sum_{s'} p[s'|s, a] \cdot v_{\pi}(s') \right)\end{aligned}$$

This is called the **Bellman equation**. For a complete proof see https://jmichaux.github.io/_notebook/2018-10-14-bellman/

Iterative policy evaluation

The recursive formulation of the Bellman equation can be used to derive an iterative approximation of the value function v_π for a given policy π :

Iterative policy evaluation

Input: π , the policy to be evaluated.

Initialize an array $V(S) = 0, \forall s \in \mathcal{S}$

repeat

$\Delta \leftarrow 0$

for $s \in \mathcal{S}$ **do**

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p[s', r|s, \pi(s)](r + \gamma V(s'))$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

end for

until $\Delta < \epsilon$ (small value) **return** V



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions

- States are $\mathcal{S} = \{1, \dots, 14\}$
- Actions are $\mathcal{A} = \{\text{left}, \text{right}, \text{up}, \text{down}\}$
- rewards are -1 for all transitions until an endpoint is reached.
- the algorithm converges to V_{π} .

Random Policy

w.r.t. V_k

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

←	←	←	←
←	←	←	←
←	←	←	←
←	←	←	←

random policy

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

←	←	←	←
↑	←	←	←
←	←	←	↓
←	←	→	←

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

←	←	←	←
↑	←	←	↓
←	←	→	↓
←	→	→	←

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

←	←	←	←
↑	←	←	↓
↑	→	→	↓
→	→	→	←

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

←	←	←	←
↑	←	←	↓
↑	→	→	↓
→	→	→	←

optimal policy

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

←	←	←	←
↑	←	←	↓
↑	→	→	↓
→	→	→	←

The Bellman equation can also be used to provide an iterative approximation of the policy.

Note that once the optimal value function v_* is estimated, it is trivial to devise an optimal policy

$$\pi_*(s) = \operatorname{argmax}_a v_*(s, a)$$

Dynamic programming uses alternate sweeps of **policy evaluation** and **policy improvement** using the Bellman equation, to learn the optimal policy:

$$\pi_0 \rightarrow v_{\pi_0} \rightarrow \pi_1 \rightarrow v_{\pi_1} \rightarrow \cdots \rightarrow \pi_* \rightarrow v_*$$

Dynamic programming: Policy iteration

A. Policy evaluation sweep

```
repeat
   $\Delta \leftarrow 0$ 
  for  $s \in \mathcal{S}$  do
     $v \leftarrow V(s)$ 
     $V(s) \leftarrow \sum_{s',r} p[s', r|s, \pi(s)](r + \gamma V(s'))$ 
     $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
  end for
until  $\Delta < \epsilon$  (small value)
```

Dynamic programming: Policy iteration

A. Policy evaluation sweep

[algorithm]

B. Policy improvement sweep

repeat

 policy-stable $\leftarrow$ *true*

for $s \in \mathcal{S}$ **do**

 old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p[s', r|s, a](r + \gamma V(s'))$

if old-action $\neq \pi(s)$ **then**

 policy-stable $\leftarrow$ *false*

end if

end for

if policy-stable is false **then**

 goto A.

end if

until policy-stable

return V and π

- ▶ One issue with policy iteration is that the policy evaluation stage is itself an expensive iterative computation.

- ▶ One issue with policy iteration is that the policy evaluation stage is itself an expensive iterative computation.
- ▶ Value iteration is an optimisation on policy iteration that does not require full policy evaluation at each step:

Dynamic programming: Value iteration

- ▶ One issue with policy iteration is that the policy evaluation stage is itself an expensive iterative computation.
- ▶ Value iteration is an optimisation on policy iteration that does not require full policy evaluation at each step:

Value iteration

Initialize value array V (eg, $V(s) = 0, \forall s$)

repeat

$v \leftarrow V(s)$

$V(s) \leftarrow \max_a \sum_{s',r} p[s', r|s, a](r + \gamma V(s'))$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \epsilon$ (small value)

return a deterministic policy $\pi \simeq \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a r(s, a) + \gamma \sum_{s'} (p[s'|s, a] \cdot V(s'))$$

This algorithm for value iteration combines at each iteration policy evaluation and policy improvement.

Summary on Dynamic Programming

- ▶ Efficient: Value iteration is guaranteed to find the optimal policy in polynomial time, even though the space is $|\mathcal{A}|^{|\mathcal{S}|}$.
- ▶ Linear programming can be used to solve DP instead of value iteration, but typically cannot handle large number of states $|\mathcal{S}|$ (while value iteration can).
- ▶ One limitation of DP is that it requires full and accurate knowledge of the environment.

Outline

- 1 The k -armed bandit
- 2 Dynamic Programming
 - Closed loop systems
 - Markov decision processes
 - Estimating value functions
- 3 Temporal-difference learning
 - On-policy learning: SARSA
 - Off-policy learning: Q-learning
- 4 Deep reinforcement learning

Temporal-difference learning

- ▶ One essential limitation of Dynamic Programming is that it requires a complete knowledge of the environment dynamics.
- ▶ Temporal-difference approaches allow to learn policy from experience only: as the system interacts with its environment it improves its behaviour by updating its estimated payoff function $Q(s, a)$, to approximate the optimal payoff $q_*(a)$.
- ▶ Temporal difference algorithms can be separated into:
 - ▶ **on-policy**, where the estimate of Q is influenced by the policy of choice, and
 - ▶ **off-policy**, where the estimation of Q is irrespective of policy choice.

Estimating a policy value function v_π : TD(0)

The simplest TD algorithm can be used to estimate the value function for a given policy.

TD(0) for estimating v_π

Input: The policy π to be evaluated.

Initialize $V(S)$ arbitrarily (eg, $V(S) = 0, \forall S \in \mathcal{S}$)

for all episodes **do**

Initialize S

for all steps in episode **do**

$A \leftarrow \pi(S)$

Take action A and observe R, S'

$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

end for

end for

- ▶ In the previous algorithm, the value function is estimated iteratively using the following update:

$$V(S) \leftarrow V(S) + \alpha [R + \gamma V(S') - V(S)]$$

- ▶ Note that the difference $R + \gamma V(S') - V(S)$ is the difference between the estimates of $V(S)$ before and after the update. This is called the **TD-error**, and is central to many reinforcement algorithms.
- ▶ TD methods learn their estimates on the basis of other estimates, this is called **bootstrapping**.
- ▶ These methods come with an on-line formulation, which is appropriate for an active agent.
- ▶ Crucially, for any fixed policy π , this TD algorithm has been proven to converge to v_π .

State-Action-Reward-State-Action (SARSA):

On policy learning

- ▶ In practice, we want an algorithm for controlling an active agent, not just for estimating the value function—this means we want to update the action-value estimate $Q(s, a)$
- ▶ SARSA is an **on policy** algorithm designed for this purpose: it learns by optimising the agent's actions.
- ▶ This is done by estimating the Q-value $Q(S_t, A_t)$
- ▶ The update rule is derived from TD(0), but applied to the state-action as follows:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

SARSA

Initialize $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily and $Q(\text{terminal-state}, \cdot) = 0$.

for Each episode **do**

 Initialize S

 Choose A from S using policy derived from Q (e.g., ϵ -greedy).

for Each step of episode **do**

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ϵ -greedy).

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

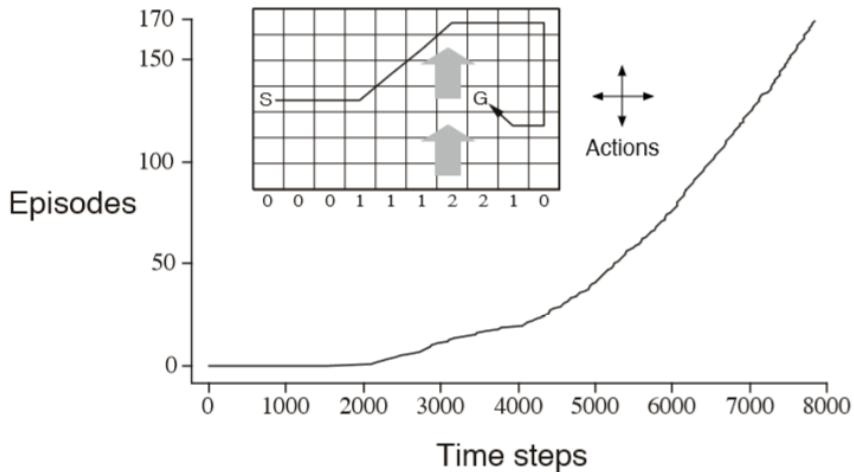
$S \leftarrow S'; A \leftarrow A'$

end for

end for

- ▶ In this algorithm we iteratively estimate q_π for the policy π and simultaneously update π to greedily optimise q_π .
- ▶ Asymptotic convergence is guaranteed if the policy tend to the greedy policy in the long run (eg, if $\epsilon = \frac{1}{t}$).

Example: Windy gridworld



- ▶ Q-learning is an **off policy** algorithm: it approximates the optimal action-value function q_* irrespective of the policy.
- ▶ The policy only impacts the algorithm by selecting the state-action pairs that are visited.
- ▶ To remove dependence on the policy, the update rule is as follows:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Q-learning

Initialize $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily and $Q(\text{terminal-state}, \cdot) = 0$.

for Each episode **do**

Initialize S

for Each step of episode **do**

Choose A from S using policy derived from Q (e.g., ϵ -greedy).

Take action A , observe R, S'

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$

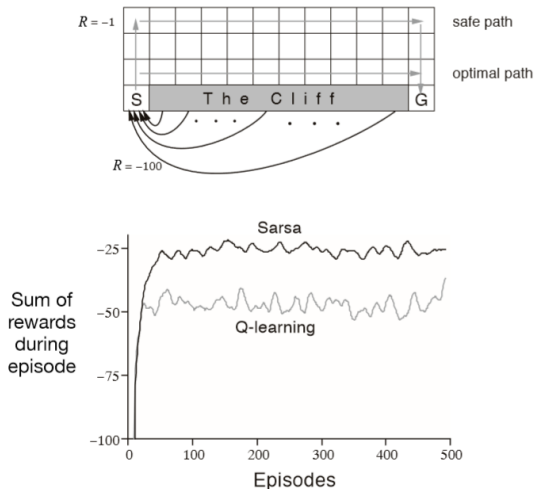
$S \leftarrow S'$

end for

end for

Example: Cliff walking

- ▶ Simple gridworld task with start (S) and goal (G) states.
- ▶ reward: -1 for all transitions
- ▶ cliff rewards -100 and returns the agent to the start.
- ▶ Q-learning finds the optimal strategy, but achieves lower reward due to ϵ -greedy policy.
- ▶ SARSA learns safer, but longer path.



Outline

- 1 The k -armed bandit
- 2 Dynamic Programming
 - Closed loop systems
 - Markov decision processes
 - Estimating value functions
- 3 Temporal-difference learning
 - On-policy learning: SARSA
 - Off-policy learning: Q-learning
- 4 Deep reinforcement learning

Question?

How can we scale these algorithms to more generic problems?

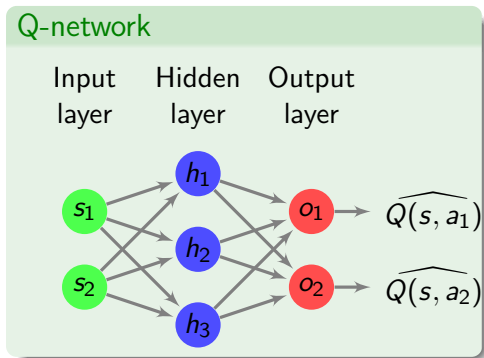
- ▶ In both SARSA and Q-learning, we are updating the value estimate using a table with $|\mathcal{S}| \times |\mathcal{A}|$ entries.
- ▶ In many cases, there may be too many states to encode effectively—the state space can even be continuous!
- ▶ One solution to overcome this limitation is to replace the table with, eg, a neural network.

Introducing the Q-network

- ▶ Fundamentally, Q is a mapping $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$
- ▶ We can parametrize it like any classification/regression problem: $Q(s, a; \theta)$

As a neural network:

- ▶ Inputs can be the (continuous) state variables $s_1, s_2, \dots, s_n$.
- ▶ Each output neuron can regress the Q value for one action a .



Training the Q-network

Remember, the Q-learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

We can replace this operation by a network weight update based on standard backpropagation.

Training the Q-network

Remember, the Q-learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

We can replace this operation by a network weight update based on standard backpropagation.

Q-network loss function

We define a differentiable, off-policy loss function based on the Bellman equation:

$$L_t = \left(R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta) - Q(S_t, A_t; \theta) \right)^2$$

Stabilising the Q-loss

One issue with the loss we have just defined is that the network we are updating is **also used as target for calculating the loss**:

$$L_t = \left(R_{t+1} + \gamma \max_a Q(S_{t+1}, a; \theta) - Q(S_t, A_t; \theta) \right)^2$$

This can lead to poor convergence of the network.

Freezing Target Network

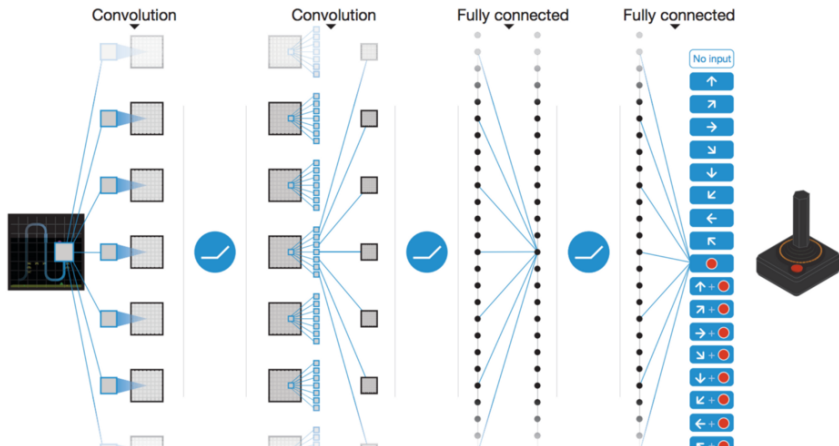
Learning convergence of the Q-network can be improved significantly by duplicating the Q-network and holding the weights of the target network $\hat{Q}$ constant while updating the Q-network, only updating the target network at longer intervals.

$$L = \sum \left(R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - \widehat{Q_T(S_t, A_t)} \right)^2$$

Deep Q-learning

Deep Q-network (DQN)

We can replace the simple, one layer neural network with a deep neural network for estimating the Q function.



Experience replay

Training a deep neural network requires more iterations than a standard table update, in order to speed-up training, we **store the agent's experiences**, allowing us to replay them as batches for training the network.

DQN: Experience replay

Experience replay

Training a deep neural network requires more iterations than a standard table update, in order to speed-up training, we **store the agent's experiences**, allowing us to replay them as batches for training the network.

In practice:

- ▶ The agent acts in the environment using current policy and Q-value estimates, gathering $(S_t, A_t, R_{t+1}, S_{t+1})$ tuples and adding them to the dataset $\mathcal{D}$;
- ▶ Replace old experiences when $|\mathcal{D}|$ becomes too large; and
- ▶ train the Q function from batches of examples sampled randomly from $\mathcal{D}$, using back-propagation.

Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A, Veness, Joel, Bellemare, Marc G, Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K, Ostrovski, Georg, et al. Human level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Beyond DQN: Actor critic models

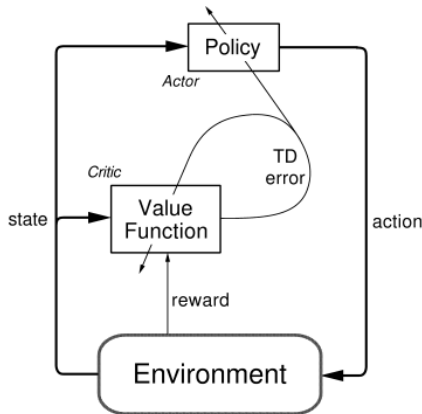
Problem

Deep Q-learning has one important flaw: it is limited to discrete action domains.

Idea

Train a pair of deep models:

- ▶ The **actor** π , which models the policy, a probabilistic mapping from state to action $\pi : \mathcal{S} \rightarrow \mathcal{A}$.
- ▶ The **critic** Q , which estimates the expected return of an action in a given state $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$.



Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver & Daan Wierstra.

Continuous control with deep reinforcement learning. *ICLR'2016*.

Markov Decision Processes

- ▶ The problem in reinforcement learning is how to select actions that maximize future rewards in the long run.
- ▶ This problem can be formalised as a Markov Decision Process (MDP).

Dynamic Programming

- ▶ DP is an efficient algorithm for solving the RL problem when state transition dynamics are fully known.
- ▶ It optimises the value function estimate by alternating policy evaluation and policy improvement.

Temporal Difference

- ▶ When the state dynamics are not known, you need TD.
- ▶ TD algorithms estimate the action-value function iteratively either using an on-policy or off-policy approach.